

자료구조 및 알고리즘

2024 년 1 학기 과제

학번 20210811 이름 이상엽

아래의 내용 중 가장 왼쪽의 열은 원래의 배열 모습이다. 그리고 가장 오른쪽 열은 정렬 이후의 배열 모습이다. 다른 열은 수업시간에 배웠던 정렬 알고리즘들 중 하나를 수행하는 중간 특정 시점의 배열의 모습이다.

navy	corn	blue	wine	bark	bark	bark
plum	mist	coal	teal	blue	blue	blue
coal	coal	corn	silk	cafe	coal	cafe
jade	jade	gray	plum	coal	corn	coal
blue	blue	jade	sage	corn	dusk	corn
pink	cafe	lime	pink	dusk	cafe	dusk
rose	herb	mint	rose	gray	gray	gray
gray	gray	navy	jade	herb	herb	herb
teal	leaf	pink	navy	jade	jade	jade
ruby	dusk	plum	ruby	leaf	leaf	leaf
mint	mint	rose	pine	lime	lime	lime
lime	lime	ruby	palm	mint	mint	mint
silk	bark	silk	coal	silk	mist	mist
corn	navy	teal	corn	plum	navy	navy
bark	silk	bark	bark	navy	palm	palm
wine	wine	wine	gray	wine	pine	pine
dusk	ruby	dusk	dusk	pink	pink	pink
leaf	teal	leaf	leaf	ruby	plum	plum
herb	rose	herb	herb	rose	rose	rose
sage	sage	sage	blue	sage	ruby	ruby
cafe	pink	cafe	cafe	teal	sage	sage
mist	plum	mist	mist	mist	silk	silk
pine	pine	pine	mint	pine	teal	teal
palm	palm	palm	lime	palm	wine	wine
----	----	----	----	----	----	----
Original	(A)	(B)	(C)	(D)	(E)	Sorted

1. 각 열의 하단에 가장 적절한 알고리즘을 찾아 이름을 기입하라.

[보기]	Quicksort	Insertion	Heapsort	Selection	Bubble
------	-----------	-----------	----------	-----------	--------

(A) : Quicksort (D) : Selection

(B) : Insertion (E) : Bubble

(C) : Heapsort

위 알고리즘의 답에 대한 이유에 대해 생각해보고, 이를 바탕으로 Original 배열, Sorted 배열과 정렬 중간의 배열을 토대로 정렬 알고리즘을 찾는 함수를 유사코드로 작성하라. 입력 : Original 배열, Sorted 배열과 정렬 중간의 배열 출력 : 사용된 정렬 알고리즘

```
def identify_sorting_algorithm(original, intermediate, sorted_array):
    if intermediate == sorted_array:
        return "Sorted"

    # Check for Bubble Sort
    if all(intermediate[i] <= intermediate[i+1] for i in range(len(intermediate)-1)):
        return "Bubble Sort"

    # Check for Insertion Sort
    if all(intermediate[i] <= intermediate[i+1] for i in range(len(intermediate)-1)):
        return "Insertion Sort"

    # Check for Selection Sort
    for i in range(len(intermediate)):
        min_index = i
        for j in range(i+1, len(intermediate)):
            if intermediate[j] < intermediate[min_index]:
                min_index = j
        if intermediate[i+1] == sorted_array[i+1]:
            return "Selection Sort"

    # Check for Heapsort
    if is_heap(intermediate):
        return "Heapsort"

    # Default to Quicksort
    return "Quicksort"

def is_heap(array):
    # Helper function to check if an array is a max-heap
    n = len(array)
    for i in range(n // 2):
        if 2*i + 1 < n and array[i] > array[2*i + 1]:
            return False
        if 2*i + 2 < n and array[i] > array[2*i + 2]:
            return False

    return True
```

2 번 문제에서 작성된 유사코드의 시간복잡도와 공간복잡도를 빅오표기법으로 나타내라.

시간복잡도: $O(n)$

공간복잡도: $O(1)$